Python for Everybody

Course 3: Using Python to Access Web Data

Modules 2–5: Regular Expressions, Network Programs, Web Surfing & Web Services

Module 2 — Regular Expressions

Section 1 — Theory

1.1 What is a Regular Expression?

A regular expression (commonly abbreviated as regex or regexp) is a sequence of characters that defines a search pattern. In computing, regular expressions provide a concise and flexible mechanism for matching strings of text — whether those strings represent specific characters, words, or complex patterns of characters.

Regular expressions are written in a formal language that is interpreted by a specialised processor called the regular expression engine. They are simultaneously one of the most powerful and most cryptic tools available to a programmer. A well-crafted regex can replace dozens of lines of imperative string-manipulation code with a single expressive pattern.

> **Key Principle:** A regular expression does not itself perform any action — it is merely a pattern. The action (search, extract, replace, split) is performed by the regex engine when the pattern is applied to a target string.

1.2 The Python re Module

Python provides native support for regular expressions through the re module, which is part of the standard library and requires no additional installation. The re module exposes a collection of functions that accept a pattern and a target string, apply the regex engine, and return results.

Function	Signature	Description
re.match()	re.match(pattern, string)	Checks for a match only at the beginning of the string. If the string does not begin with the pattern, returns None.
re.search()	re.search(pattern, string)	Searches the entire string for the first occurrence of the pattern anywhere within it. Returns a match object if found.
re.findall()	re.findall(pattern, string)	Returns a list of all non-overlapping matches of the pattern in the string. Most commonly used for extraction.
re.finditer()	re.finditer(pattern, string)	Returns an iterator yielding match objects for all non-overlapping matches. Useful when you need match position info.
re.sub()	re.sub(pattern, repl, string)	Replaces all occurrences of the pattern in the string with the replacement string repl.
re.split()	re.split(pattern, string)	Splits the string at each occurrence of the pattern and returns a list of substrings.

1.3 Common Regex Patterns (Meta-characters)

The power of regular expressions derives from a set of special characters — called meta-characters — that represent classes or positions of characters rather than literal characters themselves.

Pattern	Meaning	Example
.	Matches any single character except a newline character.	'h.t' matches 'hat', 'hot', 'hit'
^	Anchors the match to the start of the string.	'^From' matches lines beginning with 'From'
$	Anchors the match to the end of the string.	'@gmail.com$' matches strings ending with @gmail.com
\d	Matches any single decimal digit (0–9).	'\d\d\d' matches any 3-digit sequence
\D	Matches any character that is NOT a decimal digit.	'\D+' matches sequences of non-digit characters
\w	Matches any word character: letters, digits, and underscore.	'\w+' matches any word token
\W	Matches any non-word character.	'\W' matches spaces, punctuation, etc.
\s	Matches any whitespace character (space, tab, newline).	'hello\sworld' matches 'hello world'
\S	Matches any non-whitespace character.	'\S+' matches any sequence without spaces
[abc]	Matches any one of the characters a, b, or c (character class).	'[aeiou]' matches any vowel
[^abc]	Matches any character EXCEPT a, b, or c (negated class).	'[^0-9]' matches any non-digit
(abc)	Capturing group — matches the pattern abc and captures it.	'(\d+)' captures one or more digits as a group

1.4 Quantifiers

Quantifiers specify how many times the preceding element must appear for a match to succeed.

Quantifier	Meaning	Example
*	Zero or more occurrences of the preceding element.	'ab*c' matches 'ac', 'abc', 'abbc', etc.

+	One or more occurrences of the preceding element.	'ab+c' matches 'abc', 'abbc' but NOT 'ac'
?	Zero or one occurrence (makes the preceding element optional).	'colou?r' matches both 'color' and 'colour'
{n}	Exactly n occurrences.	'\d{4}' matches exactly 4 digits (e.g., year)
{n,}	n or more occurrences.	'\d{3,}' matches 3 or more digits
{n,m}	Between n and m occurrences (inclusive).	'\d{2,4}' matches 2, 3, or 4 digits

Greedy vs Lazy: By default, quantifiers are greedy — they match as many characters as possible. Append ? to a quantifier (e.g., *? or +?) to make it lazy — matching as few characters as possible. This is critical when extracting content between delimiters.

Section 2 — Practical & Coding

2.1 Comparing String Search vs Regular Expression Search

The following two code blocks perform identical tasks — scanning a mail log file for lines containing 'From:' — but use different approaches. Comparing them illustrates why regular expressions are preferred for pattern-based text processing.

Cell 1 — Traditional String Search (Without Regex)

```
hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if line.find('From:') >= 0:
        print(line)
```

Explanation:

- open('mbox-short.txt') — Opens the file and returns a file handle object. The file is read line by line using the for loop.
- line.rstrip() — Strips trailing whitespace (including the newline character \n) from each line. This is important for clean comparisons.
- line.find('From:') >= 0 — The str.find() method returns the index of the first occurrence of the substring within the string, or -1 if not found. The condition >= 0 is True whenever the substring appears anywhere in the line.
- print(line) — Prints the matching line.

Limitation: str.find() only performs literal string matching. It cannot express patterns such as 'a line that starts with From: followed by any email address'. For such pattern-based matching, regular expressions are the correct tool.

Cell 2 — Regular Expression Search (With re Module)

```
import re

hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    if re.search('From:', line):
        print(line)
```

Explanation:

- import re — Imports Python's built-in regular expression module. This must be done before any regex functions can be used.
- re.search('From:', line) — Searches the string line for the first occurrence of the pattern 'From:' anywhere in the string. Returns a match object (which evaluates to True) if found, or None (which evaluates to False) if not found. In this simple example, the result is equivalent to str.find(), but the approach scales to complex patterns effortlessly.

The key advantage becomes apparent with more complex patterns. For example, to find lines that start with 'From:' and contain an email address, you would write:

```
import re

hand = open('mbox-short.txt')
for line in hand:
    line = line.rstrip()
    # Match lines that start with 'From:' followed by an email address
    match = re.search('^From:.+@[\w.]+', line)
    if match:
        print(line)
```

Explanation of the extended pattern '^From:.+@[\w.]+':

- ^ — Anchors the match to the start of the line.
- From: — Literal text that must appear at the start.

- .+ — One or more of any character (the name portion of the email).
- @ — Literal @ symbol.
- [\w.]+ — One or more word characters or dots (the domain portion of the email).

Cell 3 — Extraction with re.findall()

```
import re


# Extract all email addresses from a file
hand = open('mbox-short.txt')
emails = list()
for line in hand:
    line = line.rstrip()
    # Pattern: word chars and dots, @, word chars and dots
    found = re.findall('[\w.]+@[\w.]+', line)
    if found:
        emails = emails + found

print(emails)
```

Explanation:

- re.findall(pattern, string) — Returns a Python list of all non-overlapping matches of the pattern in the string. If no matches are found, it returns an empty list (not None), which means it is always safe to iterate over the result.
- '[\w.]+@[\w.]+' — This pattern matches sequences of word characters and dots on either side of an @ symbol, capturing complete email addresses.
- emails = emails + found — Appends any found emails to the running list. In production code, you would typically use emails.extend(found) for efficiency.

Module 3 — Network Programs

Section 1 — Theory

1.1 The Internet Protocol Stack

To understand how Python programs communicate over a network, it is necessary to first understand the layered architecture of the internet. Modern networking is organised into a stack of protocols, where each layer provides services to the layer above it and relies on the layer below it.

The Transmission Control Protocol (TCP) is the transport-layer protocol that underpins most internet communication. Its key characteristics are:

- TCP is built on top of IP (Internet Protocol), which handles the routing of packets across networks.
- TCP adds reliability on top of IP: it detects lost packets and automatically retransmits them, ensuring that all data arrives at the destination in the correct order.
- TCP implements flow control using a transmit window mechanism, which prevents a fast sender from overwhelming a slow receiver.
- Together, TCP and IP provide a clean, reliable pipeline for application data — abstracting away the complexity of the underlying physical network (Ethernet, WiFi, Fibre, Satellite, etc.).

The protocol stack layers, from highest to lowest, are:

Layer	Protocol Examples	Responsibility
Application	HTTP, HTTPS, FTP, SMTP, DNS	User-facing protocols for web, email, file transfer, etc.
Transport	TCP, UDP	End-to-end communication, reliability, flow control
Internet	IP (IPv4, IPv6)	Addressing and routing packets between networks
Link	Ethernet, WiFi	Physical transmission over a local network segment

1.2 Port Numbers

A port number is an application-specific or process-specific software communications endpoint. It is a 16-bit integer (ranging from 0 to 65,535) that, combined with an IP address, uniquely

identifies a specific application or service running on a machine. Port numbers allow multiple networked applications to coexist simultaneously on the same physical server.

The well-known port numbers (0–1023) are assigned by the Internet Assigned Numbers Authority (IANA) to standard protocols:

Port	Protocol	Application
20, 21	FTP	File Transfer Protocol (20 = data, 21 = control)
22	SSH	Secure Shell — secure remote login and file transfer
23	Telnet	Remote login (unencrypted, rarely used today)
25	SMTP	Simple Mail Transfer Protocol — sending email
53	DNS	Domain Name System — hostname to IP resolution
80	HTTP	HyperText Transfer Protocol — web traffic
110	POP3	Post Office Protocol — receiving email
143	IMAP	Internet Message Access Protocol — email retrieval
443	HTTPS	Secure HTTP — encrypted web traffic (TLS/SSL)
3389	RDP	Remote Desktop Protocol
8080	HTTP-alt	Alternative web traffic / development servers

1.3 Sockets — The Building Block of Network Communication

A socket is a software abstraction representing one endpoint of a two-way communication link between two programs running over a network. Think of a socket as the telephone handset in a phone call — it provides the interface through which your program can send and receive data across the network.

Python's built-in socket module provides access to the Berkeley Sockets interface, the standard networking API used across virtually all operating systems. The key parameters when creating a socket are:

- **socket.AF_INET —** Address Family Internet (IPv4). Specifies that the socket will communicate using IPv4 addresses.
- **socket.SOCK_STREAM —** Stream socket. Specifies a TCP connection — data arrives in order and is guaranteed to be delivered.

1.4 Telnet — A Historical Context

Telnet is one of the oldest network protocols, allowing a user to log into a remote computer over a TCP/IP network and operate it via a command-line interface. It runs on port 23 by default.

Telnet is no longer used in production environments for two principal reasons:

1. Insecurity — Telnet transmits all data, including usernames and passwords, as unencrypted plain text. Any party able to intercept network traffic can read the credentials.
2. Replacement by SSH — Secure Shell (SSH) provides all of Telnet's functionality but with strong encryption. SSH is the modern standard for remote server access.

Historical Note: Telnet retains educational value as it allows developers to manually construct and send raw HTTP requests to web servers, making the HTTP request-response cycle visible and tangible.

1.5 HTTP — HyperText Transfer Protocol

HTTP stands for HyperText Transfer Protocol. It is the foundational protocol of the World Wide Web, defining the rules by which browsers (clients) and web servers exchange information. HTTP operates on port 80 by default (port 443 for HTTPS).

The HTTP Request-Response Cycle:

3. A user clicks a hyperlink in a browser, or a program issues an HTTP request.
4. The browser (or program) establishes a TCP connection to the web server on port 80.
5. The browser sends an HTTP GET request, specifying the URL of the desired resource.
6. The web server processes the request, locates the resource, and sends back an HTTP response containing the HTML document (or other content).
7. The browser parses the HTML and renders it for the user. Any embedded resources (images, stylesheets, scripts) trigger additional GET requests.

Important: In HTTP communication, the client must always send a request first. The server only responds — it never initiates communication. This is the fundamental request-response model.

Section 2 — Practical & Coding

2.2 Raw HTTP Request Using Python Sockets

The following code demonstrates how to make an HTTP request at the socket level — without any higher-level library. This is the lowest-level approach and is presented for educational purposes to demystify what happens under the hood when any web request is made.

Cell 1 — Complete Socket-Level HTTP Request

```
import socket

# Line 1: Import the socket module for low-level network access
# Line 3: Create a TCP socket (AF_INET = IPv4, SOCK_STREAM = TCP)
mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Line 4: Establish a TCP connection to the remote host on port 80 (HTTP)
mysock.connect(('data.pr4e.org', 80))

# Line 5: Construct the HTTP GET request as a string
# \r\n\r\n is the HTTP header terminator (two CRLF sequences)
# .encode() converts the Python string to bytes for transmission
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()

# Line 6: Send the GET request to the server
mysock.send(cmd)

# Lines 8-12: Receive the response in chunks of 512 bytes
while True:
    data = mysock.recv(512)     # Receive up to 512 bytes at a time
    if len(data) < 1:           # Empty data means the server closed the
connection
        break
    print(data.decode())        # Decode bytes back to a string and print

# Line 13: Close the socket to release the network resource
mysock.close()
```

Line-by-Line Explanation:

- **import socket —** Imports Python's built-in socket module, which provides the Berkeley Sockets interface — the low-level networking API used by virtually all operating systems.
- **socket.socket(socket.AF_INET, socket.SOCK_STREAM) —** Creates a new socket object. AF_INET specifies IPv4 addressing. SOCK_STREAM specifies TCP — a connection-oriented, reliable, byte-stream protocol.

- **mysock.connect(('data.pr4e.org', 80)) —** Establishes a TCP connection to the host 'data.pr4e.org' on port 80. This performs the TCP three-way handshake (SYN, SYN-ACK, ACK) behind the scenes.
- **cmd = 'GET ... HTTP/1.0\r\n\r\n'.encode() —** Constructs the raw HTTP GET request. The \r\n\r\n at the end is mandatory in the HTTP protocol — it signals the end of the request headers. The .encode() method converts the Python string to a bytes object, because sockets transmit raw bytes, not strings.
- **mysock.send(cmd) —** Transmits the encoded HTTP request bytes over the established TCP connection to the server. This is the critical point mentioned in the notes: in HTTP, we must send data first before we can receive any response.
- **mysock.recv(512) —** Receives up to 512 bytes of data from the server. The number 512 is the buffer size — it does not mean exactly 512 bytes will be received; it means at most 512 bytes per call.
- **if len(data) < 1: break —** When the server has finished sending its response and closes the connection, recv() returns an empty bytes object (b''). The len() check detects this condition and breaks the loop.
- **data.decode() —** Converts the received bytes back to a Python string using the default UTF-8 encoding. This is the reverse of the .encode() operation applied to the request.
- **mysock.close() —** Closes the socket and releases the underlying operating system resource. Always close sockets when finished to avoid resource leaks.

What does the response look like? The server response includes HTTP response headers (status code, date, content type, etc.) followed by a blank line and then the actual content body. The headers are part of the HTTP protocol metadata — not the document itself.

Module 4 — Programs That Surf the Web

Section 1 — Theory

1.1 Character Encoding: The Foundation of Text on Computers

Before exploring how Python communicates with web resources, it is essential to understand how text is represented digitally. All text — whether English, Chinese, Arabic, or emoji — must ultimately be stored as numbers in computer memory. The mapping between characters and their numeric representations is called a character encoding.

1.2 ASCII — The Original Character Encoding

ASCII (American Standard Code for Information Interchange) was the dominant character encoding for decades. It maps 128 characters — the English alphabet (upper and lower case), digits, punctuation, and control characters — to numbers in the range 0 to 127, stored in a single byte (though only 7 bits are used).

Python's built-in ord() function returns the ASCII (or Unicode code point) numeric value of any single character:

```
>>> print(ord('H'))   # Output: 72
>>> print(ord('e'))   # Output: 101
>>> print(ord('\n'))  # Output: 10 (newline control character)
```

Limitation of ASCII: ASCII can only represent 128 characters. It was designed for English and cannot represent characters from other writing systems such as Chinese, Arabic, Devanagari, or even common European accented characters like é or ü.

1.3 Unicode — The Universal Character Standard

Unicode is the universal character encoding standard designed to solve ASCII's fundamental limitation. Its goal is to provide a unique code point for every character from every human language and writing system — including letters, symbols, emoji, mathematical notation, and historical scripts. Unicode assigns a unique code point (a number) to each character, currently covering over 140,000 characters across 159 scripts.

1.4 UTF Encoding Schemes

Unicode defines code points (the numbers), but different encoding schemes determine how those numbers are stored as bytes. The three principal UTF (Unicode Transformation Format) schemes are:

Encoding	Width	Compatibility	Typical Use
UTF-8	1–4 bytes (variable)	Fully backward-compatible with ASCII	Web pages, APIs, databases, files — the universal standard
UTF-16	2 bytes (fixed)	Not ASCII-compatible	Windows internals, Java strings, some databases
UTF-32	4 bytes (fixed)	Not ASCII-compatible	Rarely used; wastes space for predominantly ASCII content

UTF-8 is the recommended encoding for all data exchange between systems because it is compact for ASCII-dominated content, backward-compatible with ASCII, and self-synchronising (it is possible to detect encoding errors and find character boundaries reliably).

1.5 Python 3 and Unicode

One of the most significant changes between Python 2 and Python 3 was the treatment of strings. In Python 2, strings were sequences of raw bytes by default, and the developer had to explicitly mark strings as unicode (using the u prefix, e.g., u'hello'). In Python 3, all strings are Unicode by default — the str type is always a Unicode string. There is no need for a u prefix.

However, when communicating with external systems — network sockets, databases, files — the data must be converted between Python's internal Unicode representation and a specific byte encoding:

- **encode() —** Converts a Python Unicode string to bytes using a specified encoding (typically UTF-8). Used when sending data to a network socket or writing to a binary file.
- **decode() —** Converts bytes received from an external source back into a Python Unicode string. Used when receiving data from a network socket or reading from a binary file.

The Rule: Python 3 str (Unicode) ←→ encode/decode ←→ bytes. When sending: string.encode('utf-8'). When receiving: bytes_data.decode('utf-8').

1.6 urllib — The High-Level HTTP Library

Making HTTP requests using raw sockets (as demonstrated in Module 3) requires significant boilerplate code. Python's urllib library abstracts away all the socket management, HTTP protocol handling, and encoding/decoding details, providing a much simpler interface.

urllib is not a third-party library — it is part of Python's standard library and requires no installation. It provides three key sub-modules: urllib.request for opening URLs, urllib.parse for URL manipulation, and urllib.error for exception handling.

urllib vs requests: While urllib is built into Python, many developers prefer the third-party requests library for its even cleaner API. For production code, requests is often the better choice. However, urllib is always available without installation, making it ideal for environments where third-party packages cannot be installed.

1.7 Web Scraping

Web scraping is the process of programmatically extracting data from web pages. A web scraper is a program that behaves like a browser — it fetches HTML pages, extracts information from them, and may follow links to fetch additional pages.

Common use cases for web scraping include:

- Pulling social media data or public datasets that are not available via a formal API.
- Recovering your own data from a system that provides no data export functionality.
- Monitoring websites for changes, updates, or new information.
- Building a search engine by crawling and indexing the web (often called web spidering or web crawling).

Legal and Ethical Considerations: Web scraping must be conducted responsibly. Republishing copyrighted material without permission is prohibited. Violating a website's Terms of Service through scraping may result in legal liability. Always check robots.txt and the site's terms before scraping.

1.8 BeautifulSoup — HTML Parsing

BeautifulSoup (imported from the bs4 package) is a third-party Python library specifically designed for parsing HTML and XML documents. Its primary function is to convert raw HTML text into a navigable parse tree — a structured representation of the document that makes it straightforward to locate and extract specific elements.

Key capabilities of BeautifulSoup include:

- Parsing well-formed and poorly-formed HTML gracefully — it handles real-world 'broken' HTML that browsers accept but strict parsers reject.
- Navigating the parse tree using tag names, CSS selectors, or custom search functions.
- Extracting text content, attribute values, and entire sub-trees from the HTML structure.

Before using BeautifulSoup, it must be installed:

```
pip install beautifulsoup4
```

Section 2 — Practical & Coding

2.3 Using urllib for Simple HTTP Requests

Cell 1 — Retrieving a Text File with urllib

```
import urllib.request, urllib.parse, urllib.error

# urllib.request.urlopen() handles the entire socket connection,
# HTTP request/response cycle internally — no manual socket management needed
fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')

# urllib returns file-like data — iterate line by line
for line in fhand:
    # Each line arrives as bytes; decode() converts to a Python string
    # strip() removes leading/trailing whitespace including \n
    print(line.decode().strip())
```

Explanation:

- urllib.request.urlopen(url) — Opens the specified URL and returns a file-like object. This single function call internally performs all the steps we implemented manually with sockets in Module 3: DNS resolution, TCP connection, HTTP request, and response handling.
- for line in fhand — Iterates over the response line by line, in the same way you would iterate over a local file opened with open().
- line.decode() — Each line arrives as bytes. The decode() method converts it to a Python Unicode string. The default encoding is UTF-8, which is correct for most modern web content.
- .strip() — Removes trailing whitespace, including the newline character that would otherwise appear at the end of each line.

> **Comparison:** The urllib version above achieves exactly the same result as the 20-line socket program from Module 3, but in just 4 lines of code. This is the practical value of abstraction layers.

Cell 2 — Web Scraping with BeautifulSoup

```
import urllib.request, urllib.parse, urllib.error
from bs4 import BeautifulSoup
```

```
# Prompt the user for a URL to scrape
url = input('Enter a URL: ')

# Fetch the raw HTML content of the page
# .read() reads the entire response as bytes
html = urllib.request.urlopen(url).read()

# Parse the raw HTML bytes into a BeautifulSoup parse tree
# 'html.parser' is Python's built-in HTML parser (no extra install needed)
soup = BeautifulSoup(html, 'html.parser')

# Retrieve all anchor (<a>) tags from the parsed document
tags = soup('a')

# Iterate over all anchor tags and extract the href attribute
for tag in tags:
    # tag.get('href', None) safely gets the href value
    # Returns None if the tag has no href attribute
    print(tag.get('href', None))
```

Explanation:

- **from bs4 import BeautifulSoup —** Imports the BeautifulSoup class from the bs4 package (note: the package is named bs4 even though we import BeautifulSoup from it).
- **urllib.request.urlopen(url).read() —** Fetches the URL and immediately reads the entire response body as a bytes object. Unlike the previous example which iterated line-by-line, .read() loads the full content into memory at once — appropriate for HTML pages which need to be parsed as a whole.
- **BeautifulSoup(html, 'html.parser') —** Creates a BeautifulSoup object by parsing the raw HTML bytes. The second argument specifies which parser to use — 'html.parser' is Python's built-in parser. The resulting soup object represents the entire document as a navigable tree.
- **soup('a') —** A shorthand for soup.find_all('a'). Returns a list of all <a> (anchor/hyperlink) tag objects found anywhere in the document. Each element in the list is a BeautifulSoup Tag object.
- **tag.get('href', None) —** Safely retrieves the value of the href attribute from each anchor tag. The second argument (None) is the default value returned if the attribute is not present — preventing a KeyError if an <a> tag lacks an href.

Module 5 — Web Services and XML

Section 1 — Theory

1.1 The Challenge of Cross-Language Data Exchange

When two software systems need to exchange data, they face a fundamental challenge: each system may be written in a different programming language, running on a different operating system, with different internal data representations. A Python dictionary cannot be directly transmitted to a Java application, because Java does not have a native dictionary type.

The solution is to use a wire format — a standardised, language-neutral representation of data that can be transmitted over a network (hence 'wire') and interpreted by any programming language. Both XML and JSON are wire formats.

The two-stage process for data exchange using a wire format:

8. Serialisation — The process of converting a language-specific data structure (e.g., a Python dictionary) into a wire format (e.g., XML or JSON) for transmission.
9. De-serialisation — The process of converting the received wire format data back into a language-specific data structure (e.g., a Java HashMap or a Python dictionary) on the receiving end.

1.2 XML — Extensible Markup Language

XML (Extensible Markup Language) is a markup language designed specifically for storing and transporting data. Unlike HTML, which is designed for displaying data, XML is focused on describing the meaning and structure of data.

Key characteristics of XML:

10. Self-descriptive — XML tag names describe the data they contain (e.g., <name>Chuck</name> makes it immediately clear that the content is a name).
11. Custom Tags — Unlike HTML (where tags like <p>, , etc. are predefined), XML allows developers to define their own tag names, making it extensible and adaptable to any domain.
12. Hierarchical Structure — XML data is always organised as a tree. Every XML document has exactly one root element, which may contain nested child elements to arbitrary depth.
13. Platform-Independent — XML is plain text, readable by any language on any operating system.
14. Used for Data Exchange — XML is the format underlying many web services, APIs, configuration files, and enterprise integration protocols (e.g., SOAP).

XML originated as a simplified subset of SGML (Standard Generalised Markup Language) and is designed to be relatively human-legible — a deliberate design choice that makes debugging and manual inspection feasible.

Example XML Document:

```
<people>
    <person>
        <name>Chuck</name>
        <phone>303 4456</phone>
    </person>
    <person>
        <name>Noah</name>
        <phone>622 7421</phone>
    </person>
</people>
```

1.3 XML Schema (XSD)

An XML Schema (or XSD — XML Schema Definition) is itself an XML document that defines the rules for a particular class of XML documents. It specifies which elements and attributes are allowed, their data types, and the order in which elements may appear. An XSD is analogous to a contract or blueprint — it formally specifies what valid XML documents of a particular type must look like.

XSD files conventionally have the file extension .xsd. The World Wide Web Consortium (W3C) XML Schema language is the most widely used schema language, having largely superseded older approaches such as DTD (Document Type Definition) and SGML-derived schema systems.

Common XSD data types used in web services:

XSD Type	Example	Description
xs:string	<name type='xs:string'/>	Arbitrary text data
xs:date	<start type='xs:date'/>	Date in YYYY-MM-DD format
xs:dateTime	<startdate type='xs:dateTime'/>	Date and time (ISO 8601), commonly in UTC/GMT
xs:decimal	<prize type='xs:decimal'/>	Decimal numbers with arbitrary precision

xs:integer	<weeks type='xs:integer'/>	Whole numbers without a decimal point

1.4 JSON — JavaScript Object Notation

JSON (JavaScript Object Notation) is a lightweight data-interchange format that has largely supplanted XML for web API communication due to its greater simplicity and closer alignment with modern programming language data types.

Key characteristics of JSON:

- Simpler than XML — No opening/closing tags; data is represented as key-value pairs enclosed in braces {}.
- Human-readable — JSON is compact and intuitive, resembling Python dictionary syntax closely.
- Language-independent — Despite being derived from JavaScript's object literal syntax, JSON is a language-neutral format parseable by virtually every modern programming language.
- Widely adopted — JSON is the de facto standard for REST API request and response bodies, as well as configuration files.

Feature	XML	JSON
Verbosity	High — requires open and close tags for every element	Low — compact key-value notation
Human readability	Moderate — tag names add context but add noise	High — clean, minimal syntax
Complexity	Supports schemas, namespaces, attributes, comments	Simple structure; no schema standard
Primary use cases	SOAP web services, config files, document storage	REST APIs, configuration, data exchange
Python parsing	xml.etree.ElementTree module	json module (built-in)

1.5 Service-Oriented Architecture and APIs

Most non-trivial web applications do not operate in isolation — they consume services provided by other applications. This architectural approach is called Service-Oriented Architecture (SOA). An Application Programming Interface (API) is the technical mechanism through which one application exposes its services to others.

Common examples of service consumption include credit card payment processing (the merchant's application calls the payment processor's API) and hotel reservation systems (the booking website calls the hotel chain's availability API).

APIs typically communicate via HTTP(S) and use JSON or XML as the data exchange format. Many organisations offer public APIs — sometimes free, sometimes with a usage-based pricing model.

1.6 Geoapify and API Proxies

Geoapify is a location-based service (LBS) platform that provides APIs for geocoding (converting place names to latitude/longitude coordinates), maps, routing, and geographic data enrichment. Developers use it to add location intelligence to their applications.

An API proxy is an intermediary server that sits between a client application and the target API. The client's requests are routed through the proxy, which may handle authentication, rate limiting, caching, or other concerns on behalf of the client. In educational settings, a proxy is often used to allow students to access paid APIs without needing to create individual accounts or manage API keys.

Proxy chain: Your code ↔ Cloudflare ↔ py4e-data server ↔ Geoapify

Section 2 — Practical & Coding

2.4 Parsing XML with Python

Cell 1 — Parsing an XML String with ElementTree

```
import xml.etree.ElementTree as ET

# Define an XML string using triple quotes (multi-line string)
data = '''<person>
    <name>Chuck</name>
    <phone type="intl">
        +1 734 303 4456
    </phone>
    <email hide="yes"/>
</person>'''

# Parse the XML string into an ElementTree object
# ET.fromstring() parses a string and returns the root Element
```

```
tree = ET.fromstring(data)

# Navigate to the <name> element and retrieve its text content
print('Name:', tree.find('name').text)

# Navigate to the <email> element and retrieve the 'hide' attribute value
print('Attr:', tree.find('email').get('hide'))
```

Explanation:

- **import xml.etree.ElementTree as ET —** Imports Python's built-in XML parsing module. The alias ET is universally adopted by convention.
- **Triple-quoted string ('''...''') —** Triple quotes allow multi-line strings in Python. This is used here to define a multi-line XML document as an inline string literal. Triple quotes are also used for docstrings and any string content spanning multiple lines.
- **ET.fromstring(data) —** Takes an XML string and parses it into an in-memory tree structure. Returns the root Element object (in this case, the <person> element). This is the starting point for all subsequent navigation.
- **tree.find('name') —** Searches the direct children of the root element for an element with the tag name 'name'. Returns the first matching Element object, or None if not found.
- **.text —** The text property of an Element object contains the text content between the opening and closing tags of that element. For <name>Chuck</name>, .text returns 'Chuck'.
- **.get('hide') —** The get() method retrieves the value of a specified attribute from an Element. For <email hide="yes"/>, .get('hide') returns 'yes'. If the attribute does not exist, get() returns None by default (or a specified default value).

Cell 2 — Parsing JSON with the json Module

```
import json

# Define a JSON-formatted string
data = '''
{
  "name": "Chuck",
  "phone": {
    "type": "intl",
    "number": "+1 734 303 4456"
  },
  "email": {
    "hide": "yes"
```

```
  }
}'''

# Deserialise the JSON string into a Python dictionary
info = json.loads(data)

# Access values using standard Python dictionary notation
print('Name:', info['name'])
print('Hide:', info['email']['hide'])
```

Explanation:

- **import json —** Imports Python's built-in json module. No installation required.
- **json.loads(data) —** Deserialises (parses) a JSON-formatted string into the equivalent Python data structure. JSON objects {} become Python dictionaries, JSON arrays [] become Python lists, JSON strings become Python str, JSON numbers become Python int or float. This is the de-serialisation step.
- **info['email']['hide'] —** Accesses a nested value using chained dictionary key lookups. Since the JSON object has an 'email' key whose value is another object with a 'hide' key, two successive [] lookups retrieve the final value.

json.loads vs json.load: json.loads() (with 's') parses a JSON string. json.load() (without 's') parses a JSON file object. Similarly, json.dumps() serialises a Python object to a string, while json.dump() writes it to a file.

Cell 3 — Making a Geocoding API Request

```
import urllib.request, urllib.parse
import http, json, ssl

# The proxy service URL that routes to Geoapify
serviceurl = 'https://py4e-data.dr-chuck.net/opengeo?'

# Create an SSL context that disables certificate verification
# (only needed for some network environments; not recommended for production)
ctx = ssl.create_default_context()
ctx.check_hostname = False
ctx.verify_mode = ssl.CERT_NONE
```

```
while True:
    address = input('Enter location: ')
    if len(address) < 1: break

    address = address.strip()

    # Build the query parameter dictionary
    parms = dict()
    parms['q'] = address

    # Encode the parameters into a URL-safe query string
    # e.g. 'Ann Arbor, MI' becomes 'q=Ann+Arbor%2C+MI'
    url = serviceurl + urllib.parse.urlencode(parms)
    print('Retrieving', url)

    # Fetch the URL with the SSL context
    uh = urllib.request.urlopen(url, context=ctx)
    data = uh.read().decode()
    print('Retrieved', len(data), 'characters')

    # Deserialise the JSON response into a Python dictionary
    js = json.loads(data)

    # Extract latitude, longitude, and formatted address from the response
    lat  = js['features'][0]['properties']['lat']
    lon  = js['features'][0]['properties']['lon']
    location = js['features'][0]['properties']['formatted']

    print('lat', lat, 'lon', lon)
    print(location)
```

Explanation:

- **import ssl —** Imports the ssl module for managing HTTPS (SSL/TLS) connections. Required when the urllib context needs SSL configuration.
- **urllib.parse.urlencode(parms) —** Converts a Python dictionary of query parameters into a URL-encoded query string. For example, {'q': 'Ann Arbor, MI'} becomes

'q=Ann+Arbor%2C+MI'. This encoding is necessary because URLs cannot contain spaces or certain special characters — they must be percent-encoded.

- **url = serviceurl + urllib.parse.urlencode(parms) —** Constructs the complete API URL by appending the encoded query string to the base service URL. The result is a complete, valid URL that the server can parse.
- **uh.read().decode() —** Reads the entire HTTP response body as bytes and decodes it to a Python string using UTF-8. The decode step converts the raw bytes returned by the server into a Python str that json.loads() can then parse.
- **json.loads(data) —** Parses the JSON response string into a Python dictionary (js). The API returns a GeoJSON FeatureCollection object.
- **js['features'][0]['properties']['lat'] —** Navigates the nested JSON structure: 'features' is a list of location results; [0] selects the first (best) result; 'properties' contains the location attributes; 'lat' is the latitude coordinate.

Real-World Pattern: This cell encapsulates the complete pattern for consuming a REST API in Python: build URL → make HTTP request → read response → parse JSON → extract data. This pattern is used in virtually every data science and AI project that integrates external data sources.

— End of Python for Everybody Course 3 —

GenAI & Data Science Production Engineering Series